

Verteilte Systeme

Abgabe 1 - Bericht

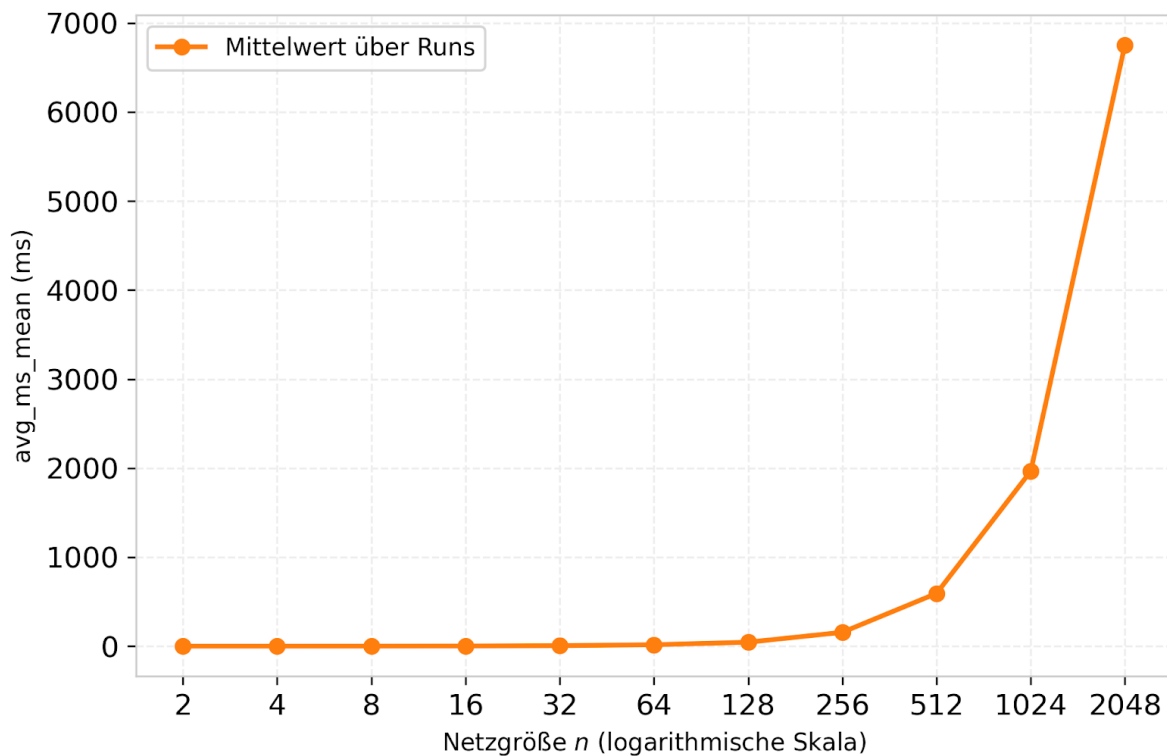
Aufgabe 1

Für die Implementierung der ersten Aufgabe habe ich mir von ChatGPT die Funktionsweise der UDP Kommunikation erklären lassen. Da keine Programmiersprache vorgegeben war, habe ich mich für Python entschieden. Ich bin bei der Entwicklung inkrementell vorgegangen, mit der Idee mir vor jedem Schritt eine Erklärung von der KI geben zu lassen, welche Optionen ich für die mögliche Umsetzung habe. Zu Beginn habe ich mir ein Codegerüst mit GPT generieren lassen, welches den UDP-Socket definiert, eine Funktion zum senden von UDP-Unicast-Nachrichten und eine Schleife, die auf Nachrichten wartet und im Anschluss auch verschickt. Um das ganze zu starten, habe ich ein weiteres Skript geschrieben, welches die Anzahl an gewünschten Knoten über eine Eingabe erhält und entsprechend viele Prozesse startet, die jeweils ein Knoten im Ring sind. Jeder Prozess läuft parallel auf demselben Gerät und erhält eine ID. Mithilfe der ID wird der UDP-Port in jedem Prozess über $port = base_port + process_id$ festgelegt. Anschließend habe ich den Multicast implementiert, der durch ein Multicast-Socket realisiert wird. Als eine weitere Eingabe wird die Wahrscheinlichkeit p zum Auslösen des Multicast vom Nutzer übergeben. Als letzten Schritt habe ich die Zustände für die Statistik und die Terminierung hinzugefügt. Nun wird auch k eingelesen, was die Anzahl an Runden ohne Multicast definiert, welche zur Terminierung führen. Der Prozess P0 ist dafür verantwortlich zu prüfen, ob die Anwendung terminiert oder nicht. Dafür wird die Variable *silent_rounds* mit k verglichen. Wenn der Grenzwert erreicht ist, wird verschickt P0 eine Nachricht vom Type „STOP“. Die anderen Knoten rechnen die versendeten Multicasts zusammen und schicken die Information an den nächsten Knoten. Danach wird der Prozess beendet. Wenn P0 den STOP-Token bekommt, gibt er die Zusammenfassung raus und terminiert das System vollständig. Anfangs habe ich *silent_rounds* zurückgesetzt, wenn P0 eine Multicast-Nachricht erhalten hat. Im Laufe der Entwicklung ist mir aufgefallen, dass es zu Inkonsistenzen kommen könnte. P0 könnte entscheiden, dass der Grenzwert erreicht ist, obwohl noch eine Multicast-Nachricht unterwegs ist, wodurch das System fälschlicherweise terminiert. Um das zu beheben, habe ich die Information stattdessen im Token gespeichert. Wenn ein Multicast stattfindet, wird die Boolean *fired* auf *true* gesetzt und dann wird der Token erst an den nächsten Knoten geschickt. Die Information, ob es ein Feuerwerk gab, wird von P0 aus dem Token gelesen und darauf basierend *silent_rounds* das System fortgesetzt oder beendet. Wenn es nicht beendet wird, wird *fired* auf *false* zurückgesetzt und es geht weiter.

Da die Ergebnisse nicht deterministisch sind habe ich, für jedes n drei Durchläufe machen lassen. Die Messwerte zeigen, dass die Anzahl an durchlaufenen Runden nur gering steigt, bei Verdopplung der Knoten. Das ist logisch, da mehr Knoten eine chance haben einen Multicast auszulösen, auch bei einem kleinem p . Die Anzahl an totalen Multicast ist linear zum

wachsendem n und immer sehr nah am Wert von n . Die durchschnittliche Zeit für eine Runde wächst mit steigender Knotenanzahl. Ab $n = 8$ erhöht sich die Durchschnittszeit für eine Runde um mehr als das Dreifache beim nächsten n . Bei 2048 Knoten beträgt die Gesamtlaufzeit 116ms. Es werden $n-1$ Nachrichten bei einem Multicast verschickt, wodurch das System bei wachsendem n länger braucht, da auch die Anzahl an Multicasts steigt.

Aufgabe 1: Rundenzeit vs. Netzgröße n



Die Ergebnisse sind unter results_aufgabe1.csv zu finden.

Aufgabe 2

Die Umsetzung von Aufgabe 2 basiert größtenteils auf dem Code aus Aufgabe 1. In Aufgabe 2 laufen die Prozesse nicht mehr auf einem gemeinsamen Rechner, sondern auf unterschiedlichen Rechnern. Deswegen gibt es eine Konfigurationsdatei (peers.txt), die die IP-Adressen und die Ports der jeweiligen Rechner beinhaltet. Diese wird beim Start eines Prozesses ausgelesen, damit die Rechner ihre Kommunikationspartner und einzelne Knoten ihren Nachfolger kennen. Der Knoten P0 wird dabei als letztes eingelesen und gestartet, damit alle anderen Knoten initialisiert sind, bevor der Token durch P0 geschickt wird. Falls Multicastim Netzwerk unterbunden wird, wurde als Fallback eine Variante mit Unicast hinzugefügt. Beim starten des Prozesses, kann man über eine Flag den gewünschten Modus auswählen.

Zuhause konnte ich es nur auf zwei Rechnern ausführen, wodurch es schwierig ist die Ergebnisse mit denen aus Aufgabe 1 zu vergleichen.

```
SUMMARY {"type": "SUMMARY", "n": 2, "k": 3, "p_start": 0.5, "fireworks": "unicast", "rounds": 4, "total_multicasts": 1, "round_time_min_ms": 0.028699985705316067, "round_time_avg_ms": 0.07434999861288816, "round_time_max_ms": 0.16039999900385737}
```

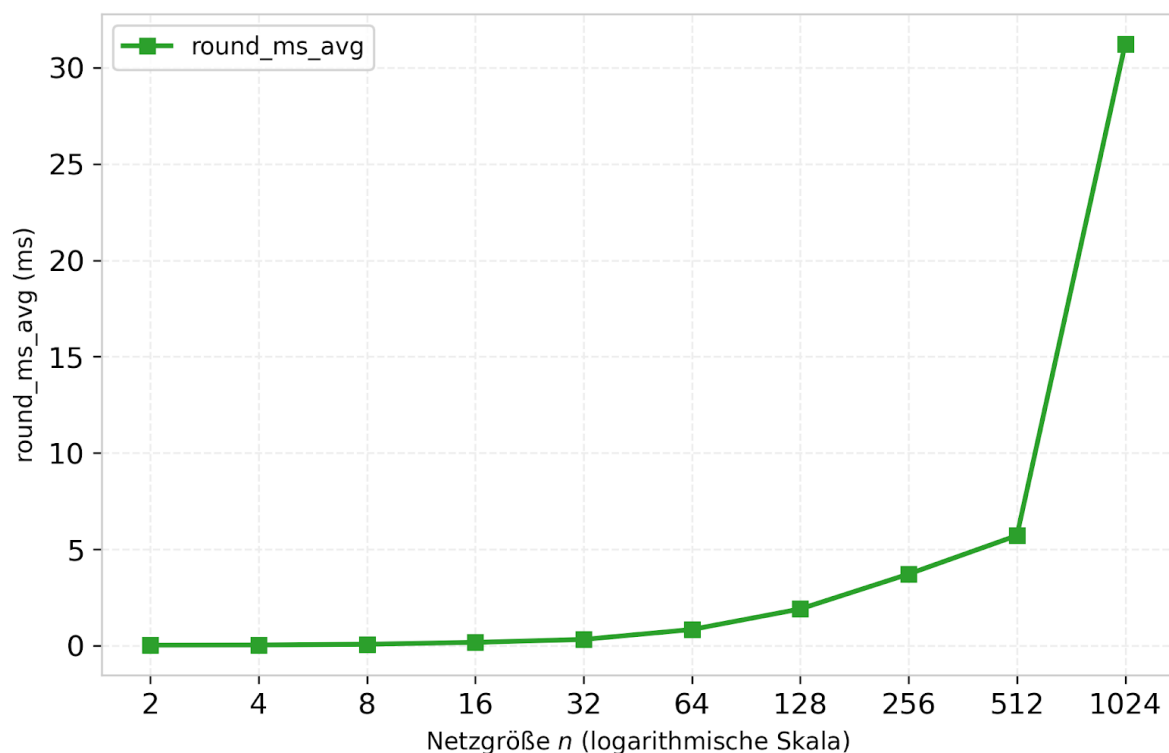
In dem Log ist aber zu sehen, dass die minimale, maximale und durchschnittliche Zeit einer Runde geringer sind, als die in Aufgabe 1. Ich hätte erwartet, dass die Kommunikation auf einem Rechner schneller wäre.

Aufgabe 3

Um Aufgabe 3 zu lösen habe ich Claude Code mit dem Model Fable 5 genutzt. Für die Umsetzung kann die Logik aus Aufgabe 1 übertragen werden. Der wichtigste Unterschied zwischen den Aufgaben ist, dass in Aufgabe 3 ein Simulator verwendet wird. Der Simulator bietet bereits grundlegende Funktionen für die Umsetzung, wie die Methoden `send()`, `sendChecked()`, `receive()` und `broadcast()`. Auch die generelle Implementierung eines Tokenrings ist in `OneRingToRuleThemAllTest` bereits gegeben. Der vorherige Main-Loop ist nun im `engage` eines Nodes.

Der Unterschied in den Messwerten ist deutlich zu sehen. Die Anzahl an Multicasts ist nun deutlich über dem Wert n . Die Zeit, welche durchschnittlich für eine Runde gebraucht wird, ist um ein vielfaches geringer, als in Aufgabe 1

Aufgabe 2: Rundenzeit vs. Netzgröße n



Aufgabe 4

Es können verschiedene Fehler im Programmablauf auftreten, die zu Inkonsistenzen führen. Konsistenzkriterien, deren Verletzungen und Lösungen sind:

Tokenintegrität:

Es gibt zu jedem Zeitpunkt nur ein Token.

Verletzungen:

- Duplikate: Es können mehrere Tokens im Umlauf sein.
- Verlust: Ein Token kann verloren gehen.

Lösungen:

Jeder Knoten hat einen Rundenzähler, der durch den Token inkrementiert wird. Das Token selbst speichert die aktuelle Runde. Wenn diese niedriger als *lastTokenRound* ist, wird das Token verworfen. So wird ein doppeltes Token vermieden. Wenn die aktuelle Runde größer ist, dann wird eine Fehlermeldung ausgegeben.

Vollständigkeit:

Jeder Knoten sieht den Broadcast eines anderen Knotens genau einmal und erhält auch jede Nachricht

Verletzung:

- Ein Prozess kann terminieren, obwohl noch eine Nachricht unterwegs ist.
- Jede Nachricht wird genau einmal beobachtet.

Lösung:

Das Token mit der Stop-Nachricht führt einen Soll-Wert mit sich. Dieser wird mit der vom Knoten beobachteten Anzahl empfangener Nachrichten verglichen. Wenn er zu niedrig ist, werden solange Nachrichten aus der Mailbox abgefragt, bis der Wert gleich ist und erst dann wird das Token verarbeitet.

Kausale Reihenfolge:

Ein Broadcast muss immer vor der Weitergabe des Tokens passieren. Das Senden einer Nachricht gehört logisch zur aktuellen Runde.

Verletzung:

- Das Token kann schneller wandern, als die Broadcast-Nachricht bei einem Knoten ankommen. Wenn der Knoten das Token erhält, ist seine Sicht inkonsistent und er erhält eine Nachricht aus der Vergangenheit.

Lösungen:

Die gleichen Maßnahmen wie bei der Vollständigkeit.

Korrekte Terminierung:

Alle Prozesse beenden die Anwendung mit derselben Sicht (Rundenzahl, Gesamtzahl an Broadcasts). Die Terminierung basiert auf einem konsistenten *silent_rounds*.

Lösung:

Wie bereits in Aufgabe 1 beschrieben, wird die Variable *silent_rounds* im Token mitgetragen und inkrementiert, wodurch sie konsistent ist und als Abbruchbedingung verwendet werden kann.

Ergebnisse und Anzahl der Verletzungen sind in der Datei `results_aufgabe3.csv` zu finden.